MODELOS DE LINGUAGEM E GENERATIVOS

Modelos Especializados, Fine-Tuning & RAG

Duas Estratégias Complementares de Adaptação

PARTE 1: MODELOS ESPECIALIZADOS (FINE-TUNING)

- Por que especializar? Modelo de Fundação vs. Especialista
- Supervised Fine-Tuning (SFT) & Instruction Tuning
- Parameter-Efficient Fine-Tuning (PEFT): Adapters, LoRA e QLoRA
- Alinhamento de Preferências: RLHF e DPO

PARTE 2: RETRIEVAL-AUGMENTED GENERATION (RAG)

- Por que RAG? Mitigação de Alucinações e Dados Dinâmicos
- Arquitetura em 2 Fases: Retrieval & Grounded Generation
- Chunking, Embeddings e Bancos de Dados Vetoriais (FAISS/Chroma)

Por Que Especializar? Transição do Modelo Base para o Especialista

Modelo de Fundação (Base)

Conhecimento Genérico: Pré-treinado em terabytes de dados não rotulados da Web.

Completude de Texto: Prevê o próximo token, mas não sabe responder perguntas diretamente.

Comportamento Não-Alinhado: Pode continuar textos de forma imprevisível ou indesejada.

O Gargalo do Domínio

Vocabulário Específico: Jargões jurídicos, médicos ou corporativos ausentes no treino.

Formato de Resposta: Necessidade de saídas estruturadas (JSON, SQL, Markdown formal).

Privacidade & Custos: Modelos gigantes de API são caros e violam compliance de dados internos.

Modelo Especializado

Ajuste Fino Supervisionado: Treinado em pares instrução-resposta de alta qualidade.

Alta Eficiência: Modelos menores (1B-7B) superam modelos genéricos de 70B na tarefa focada.

Hospedagem Privada: Roda em infraestrutura local (On-Premise / Nuvem Privada).

Ciclo de Vida do Treinamento: Pré-Treino, SFT e Preference Tuning

1. Pré-Treinamento (Pretraining)

Dados: Terabytes de texto não rotulado (Self-Supervised Learning).

Objetivo: Minimizar a perda de previsão do próximo token.

Custo: Milhões de dólares em clusters de milhares de GPUs.

Resultado: Base Model (ex: Llama-3-Base).

2. Instruction Tuning (SFT)

Dados: Milhares de pares curados {Instrução, Resposta}.

Objetivo: Ensinar o modelo a seguir comandos e atuar como assistente.

Custo: Acessível (poucas horas em 1-8 GPUs).

Resultado: Instruct Model (ex: Llama-3-Instruct).

Aula 3

3. Preference Tuning (DPO/RLHF)

Dados: Pares de comparações {Aprovado, Rejeitado}.

Objetivo: Alinhar comportamento com preferências humanas de utilidade e segurança.

Técnicas: RLHF (Reward Model) ou DPO (Direct Preference Optimization).

Resultado: Aligned Chat Model final.

Supervised Fine-Tuning (SFT): Estruturação de Dados e Modelagem

Como Funciona o SFT?

Formatação de Templates: Os dados de treino usam tags especiais para delimitar o papel do usuário e do assistente (ex: <|user|> ... <|end|> <|assistant|> ...).

Cálculo de Perda Seletivo: A perda (Cross-Entropy Loss) é calculada predominantemente sobre os tokens de resposta do assistente, não sobre o prompt de instrução.

Generalização de Instruções: Treinar em milhares de tarefas variadas (resumos, classificação, código) permite responder a novas instruções não vistas no treino.

```
# Exemplo de Registro para Instruction Tuning
[
  {
    'instruction': 'Classifique o sentimento do filme.',
    'input': 'The movie was all portent and no content.',
    'output': 'Negative Review'
  },
  {
    'instruction': 'Extraia as entidades mencionadas.',
    'input': 'O paciente foi atendido no Hospital Sirio em SP.',
    'output': 'Hospital: Sirio, Local: SP'
  }
]
```

Full Fine-Tuning vs. PEFT: O Problema da Memória de VRAM

Full Fine-Tuning (Atualização Total)

Atualiza TODOS os pesos do modelo pré-treinado ($W = W_0 + \Delta W$).

Consumo Extremo de VRAM: Um modelo de 7B parâmetros exige ~112 GB de VRAM para armazenar pesos, gradientes e estados do otimizador (AdamW).

Esquecimento Catastrófico (Catastrophic Forgetting): O modelo pode perder suas capacidades gerais ao hiper-ajustar em um conjunto pequeno.

Armazenamento Pesado: É necessário salvar uma cópia completa de 14 GB a 30 GB de pesos para cada nova tarefa.

PEFT (Parameter-Efficient Fine-Tuning)

Mantém os pesos do modelo base CONGELADOS (W_0).

Treina apenas um conjunto minúsculo de novos parâmetros (< 1% do total).

Redução Drástica de VRAM: Permite fine-tuning de modelos de 7B/13B em GPUs de consumo de 16GB–24GB (ex: RTX 3090 / T4).

Módulos Leves: O arquivo de pesos treinado (adapter) ocupa apenas alguns megabytes (10MB–100MB).

Multi-Especialização: É possível trocar os adapters leves em tempo real sobre o mesmo modelo base.

Low-Rank Adaptation (LoRA): Decomposição de Matrizes de Posto Baixo

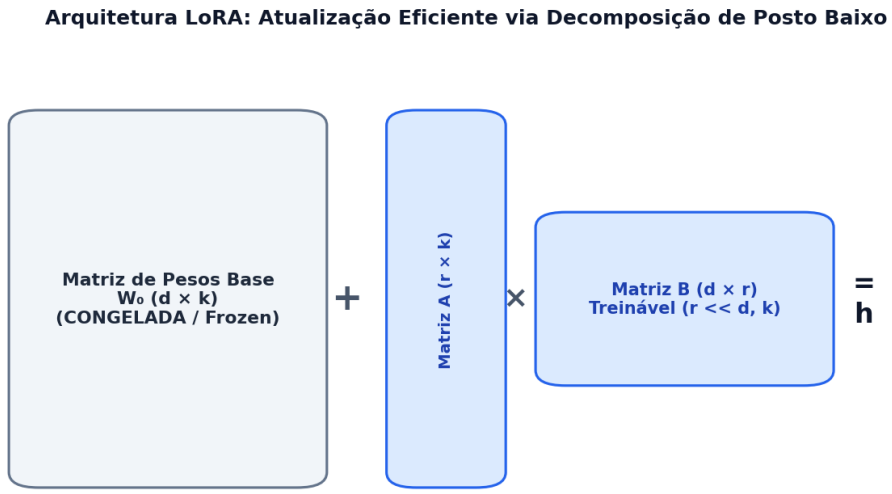
Fundamentos do LoRA (Hu et al., 2021)

Hipótese do Posto Intrínseco: A mudança de pesos ΔW durante a adaptação tem uma dimensão intrínseca muito menor que a matriz original W_0.

Fatoração Matricial: Representa ΔW = B × A, onde A é (r × k) e B é (d × r), com rank r << min(d, k) (ex: r=8 ou r=16).

Sem Latência Adicional: Na inferência, as matrizes B × A podem ser somadas diretamente aos pesos W_0, sem custo computacional extra.

Alvos Principais: Aplicado tipicamente às projeções de atenção (W_Q, W_V ou W_K, W_O).



QLoRA: Quantização de 4-Bits & Fine-Tuning Extremamente Eficiente

O que é QLoRA (Dettmers et al., 2023)?

Quantização de 4-Bits (NF4): O modelo base é comprimido de 16-bit (FP16) para o tipo NormalFloat 4-bit, reduzindo o uso de memória em ~75%.

Double Quantization (DQ): Quantiza os próprios fatores de escala de quantização, economizando memória adicional.

Paged Optimizers: Gerencia picos de VRAM durante o treinamento transferindo memória para a RAM quando necessário.

Precisão Mantida: Permite fine-tuning de modelos de 65B em GPUs de 48GB ou modelos de 7B em GPUs de 10GB sem perda mensurável de qualidade.

```
# Configuração QLoRA com bitsandbytes & PEFT
from transformers import BitsAndBytesConfig
from peft import LoraConfig

# 1. Quantização 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.float16
)

# 2. Configuração LoRA
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=['q_proj', 'v_proj'],
    lora_dropout=0.05,
    task_type='CAUSAL_LM'
)
```

Alinhamento com Preferências Humanas: RLHF vs. DPO

RLHF (Reinforcement Learning from Human Feedback)

Modelo de Recompensa (Reward Model): Treina um modelo separado para dar pontuação de aprovação a respostas geradas.

Otimização PPO (Proximal Policy Optimization): Ajusta a política do LLM para maximizar a recompensa sem se afastar demais do modelo base.

Complexidade: Requer manter 4 modelos simultâneos na VRAM (Base, Actor, Critic, Reward).

DPO (Direct Preference Optimization)

Elimina o Reward Model: Formula o alinhamento de preferências diretamente como uma função de perda de classificação sobre os dados {Aprovado, Rejeitado}.

Estabilidade e Simplicidade: Não requer aprendizado por reforço nem PPO instável.

Eficiência de VRAM: Treina diretamente o modelo com a biblioteca `trl` (`DPOTrainer`).

Padrão Atual: Utilizado no alinhamento das principais famílias open-source recentes.

Acesse: Aula_05_Modelos_Especializados_O_Pulo_do_Gato.ipynb

Por Que RAG? Expandindo a Memória do LLM sem Re-Treinamento

1. Mitigação de Alucinações

Limitação de Memória Paramétrica: LLMs tendem a gerar respostas plausíveis porém falsas para fatos específicos.

Fundamentação (Grounding): O RAG força o modelo a responder estritamente com base em documentos reais fornecidos no contexto.

2. Dados em Tempo Real

Data de Corte (Cutoff): O conhecimento pré-treinado do LLM é congelado no momento do treino.

Atualização Instantânea: Novos documentos corporativos ou notícias podem ser adicionados ao banco de dados vetorial sem re-treinar o LLM.

3. Rastreabilidade & Citações

Caixa Preta do Modelo: Em LLMs puros é impossível saber de onde veio uma afirmação específica.

Auditoria de Respostas: O RAG permite exibir as fontes e trechos exatos (citations) que sustentaram a resposta gerada.

PARTE 2 • ARQUITETURA RAG

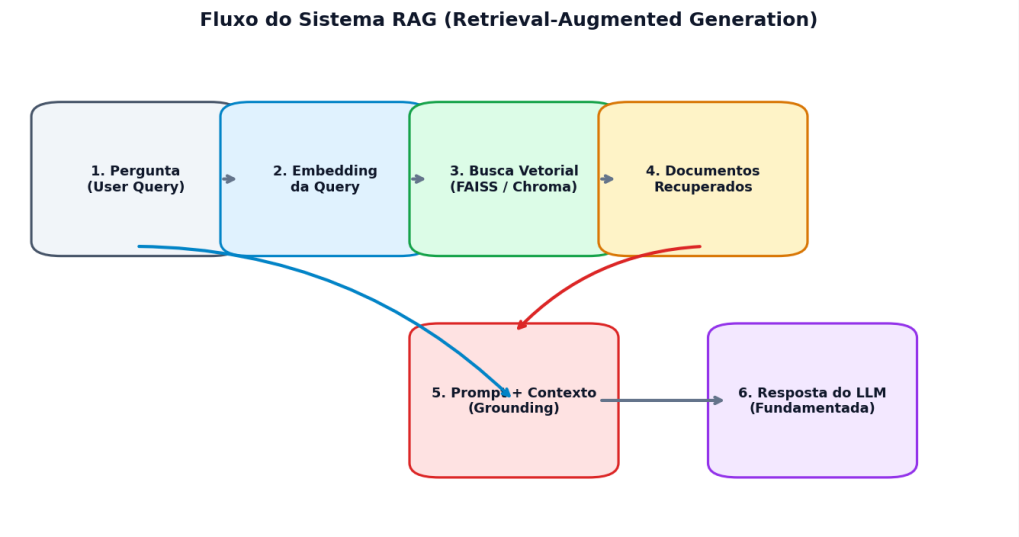
O Fluxo RAG em Duas Fases: Retrieval & Grounded Generation

As Duas Etapas Principais (Lewis et al., 2020)

1. Fase de Recuperação (Retrieval): A pergunta do usuário é convertida em um embedding vetorial e comparada via similaridade de cosseno contra um Banco de Dados Vetorial para extrair os top-K trechos relevantes.

2. Fase de Geração (Grounded Generation): Os trechos recuperados são injetados no prompt do LLM como contexto de apoio. O LLM formula a resposta ancorado estritamente nesse contexto.

Separação de Papéis: O modelo de embedding (ex: SentenceTransformers) NUNCA é lido diretamente pelo LLM; ele serve apenas para selecionar os textos em linguagem natural!



Fase 1: Divisão em Chunks, Embeddings Vetoriais e Indexação

1. Chunking de Texto

Fracionamento: Documentos longos são divididos em blocos menores (ex: 200 a 500 palavras).

Overlap (Sobreposição): Adiciona sobreposição entre blocos adjacentes para evitar perda de contexto nas fronteiras de corte.

2. Gerador de Embeddings

Modelos Bi-Encoder: Modelos levez especializados (ex: `all-MiniLM-L6-v2` ou `bge-small`) convertem cada chunk em um vetor denso (ex: 384D ou 768D).

Captura Semântica: Trechos com significados similares são mapeados para pontos próximos no espaço vetorial.

3. Banco de Dados Vetorial

Indexação Eficiente: Soluções como FAISS, ChromaDB, Pinecone e Qdrant usam estruturas ANN (Approximate Nearest Neighbors).

Operação em Milissegundos: Permite buscar entre milhões de documentos instantaneamente.

Fase 2: Construção do Prompt Aumentado e Injeção de Contexto

Como o Prompt Aumentado é Estruturado?

Variável {context}: O retriever recupera os top-K chunks do FAISS e os concatena em uma única string.

Variável {question}: A pergunta original enviada pelo usuário.

Instrução de Guarda (Guardrail): O prompt explicita que o modelo deve responder usando APENAS as informações do contexto e dizer 'Não sei' caso a resposta não esteja presente.

```
# Template Típico de Prompt RAG (LangChain
/ LlamaIndex)

template = '''<|user|>
Informações Relevantes de Suporte:
{context}

Com base ESTRITAMENTE nas informações
fornecidas acima,
responda de forma concisa e precisa à
seguinte pergunta:
{question}<|end|>
<|assistant|>'''

prompt = PromptTemplate(
    template=template,
    input_variables=['context',
'question']
)
```

PARTE 2 • CÓDIGO DA AULA 05

RAG na Prática: Pipeline Completo com FAISS e Models Locais

```
# Exemplo do Notebook Aula_05_RAG: Embeddings FAISS + Local LLM
from sentence_transformers import SentenceTransformer
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain import LlamaCpp

# 1. Indexação dos Documentos no FAISS
embed_model = SentenceTransformer('all-MiniLM-L6-v2')
db = FAISS.from_texts(documents_list, embed_model)

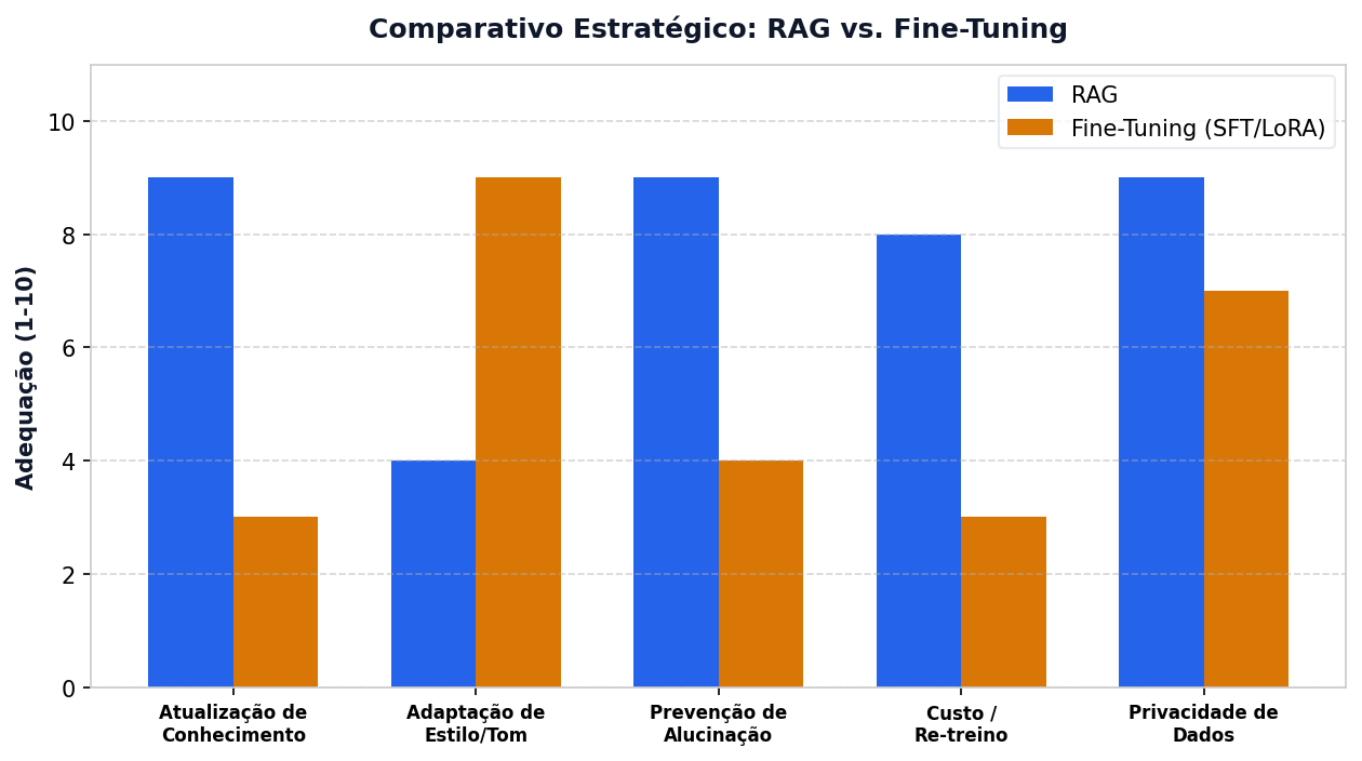
# 2. Carregamento do Gerador Local (Quantizado Phi-3 / GGUF)
llm = LlamaCpp(model_path='Phi-3-mini-4k-instruct-fp16.gguf', n_ctx=2048)

# 3. Execução da Cadeia RetrievalQA
rag_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=db.as_retriever(search_kwargs={'k': 2}),
    chain_type_kwargs={'prompt': prompt_template}
)
resposta = rag_chain.run('Qual é a política de reembolso da empresa?')
```

Acesse: Aula_05_RAG.ipynb

PARTE 2 • COMPARATIVO ESTRATÉGICO

Fine-Tuning vs. RAG: Quando Escolher Cada Abordagem?



SÍNTESE GLOBAL

Quadro Comparativo: Prompt Engineering vs. RAG vs. LoRA vs. Full Fine-Tuning

Técnica	Mecanismo	Conhecimento	Custo / VRAM	Principal Caso de Uso
Prompting / ICL	Injeção no contexto em tempo de inferência	Limitado à janela de contexto	Zero custo de treino / Latência de prompt	Protótipos rápidos e instruções simples
RAG	Busca em Vector DB + Injeção no Prompt	Dinâmico, ilimitado e atualizável	Baixo (Vector DB + Modelo de Embedding)	Bases corporativas, FAQs e busca factual
LoRA / PEFT	Treino de matrizes de rank baixo (r)	Estático (incorporado aos pesos)	Baixo (GPUs de 16GB-24GB)	Adaptação de estilo, tom e formato específico
Full Fine-Tuning	Atualização de todos os pesos (W_0)	Estático e profundo	Extremamente alto (Clusters de GPUs)	Domínios altamente especializados (ex: Bio/Code)

REFERÊNCIAS BIBLIOGRÁFICAS

Leituras Recomendadas

Alammar, J., & Grootendorst, M. (2024). Hands-On Large Language Models. O'Reilly Media. (Capítulos 4, 8 e 12).

Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv preprint arXiv:2106.09685.

Dettmers, T., et al. (2023). QLoRA: Efficient Fine-tuning of Quantized LLMs. arXiv preprint arXiv:2305.14314.

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

Rafailov, R., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.

Aula_05_Modelos_Especializados_O_Pulo_do_Gato.ipynb & Aula_05_RAG.ipynb.